

634

```
class Bulk_quote : public Quote {
    Bulk_quote* clone() const & {return new Bulk_quote(*this);}
    Bulk_quote* clone() &&
        {return new Bulk_quote(std::move(*this));}
    // 其他成员与之前的版本一致
};
```

因为我们拥有 `add_item` 的拷贝和移动版本，所以我们分别定义 `clone` 的左值和右值版本（参见 13.6.3 节，第 483 页）。每个 `clone` 函数分配当前类型的一个新对象，其中，`const` 左值引用成员将它自己拷贝给新分配的对象；右值引用成员则将自己移动到新数据中。

我们可以使用 `clone` 很容易地写出新版本的 `add_item`：

```
class Basket {
public:
    void add_item(const Quote& sale)        // 拷贝给定的对象
    { items.insert(std::shared_ptr<Quote>(sale.clone())); }
    void add_item(Quote&& sale)             // 移动给定的对象
    { items.insert(
        std::shared_ptr<Quote>(std::move(sale).clone())); }
    // 其他成员与之前的版本一致
};
```

和 `add_item` 本身一样，`clone` 函数也根据作用于左值还是右值而分为不同的重载版本。在此例中，第一个 `add_item` 函数调用 `clone` 的 `const` 左值版本，第二个函数调用 `clone` 的右值引用版本。在右值版本中，尽管 `sale` 的类型是右值引用类型，但实际上 `sale` 本身（和任何其他变量一样）是个左值（参见 13.6.1 节，第 471 页）。因此，我们调用 `move` 把一个右值引用绑定到 `sale` 上。

我们的 `clone` 函数也是一个虚函数。`sale` 的动态类型（通常）决定了到底运行 `Quote` 的函数还是 `Bulk_quote` 的函数。无论我们是拷贝还是移动数据，`clone` 都返回一个新分配对象的指针，该对象与 `clone` 所属的类型一致。我们把一个 `shared_ptr` 绑定到这个对象上，然后调用 `insert` 将这个新分配的对象添加到 `items` 中。注意，因为 `shared_ptr` 支持派生类向基类的类型转换（参见 15.2.2 节，第 530 页），所以我们可以把 `shared_ptr<Quote>` 绑定到 `Bulk_quote*` 上。

15.8.1 节练习

练习 15.30：编写你自己的 `Basket` 类，用它计算上一个练习中交易记录的总价格。

15.9 文本查询程序再探

接下来，我们扩展 12.3 节（第 430 页）的文本查询程序，用它作为说明继承的最后一个例子。在上一版的程序中，我们可以查询在文件中某个指定单词的出现情况。我们将在本节扩展该程序使其支持更多更复杂的查询操作。在后面的例子中，我们将针对下面这个小故事展开查询：

635

```
Alice Emma has long flowing red hair.
Her Daddy says when the wind blows
through her hair, it looks almost alive,
like a fiery bird in flight.
```

```
A beautiful fiery bird, he tells her,
magical but untamed.
"Daddy, shush, there is no such thing,"
she tells him, at the same time wanting
him to tell her more.
Shyly, she asks, "I mean, Daddy, is there?"
```

我们的系统将支持如下查询形式。

- 单词查询，用于得到匹配某个给定 string 的所有行：
 Executing Query for: Daddy
 Daddy occurs 3 times
 (line 2) Her Daddy says when the wind blows
 (line 7) "Daddy, shush, there is no such thing,"
 (line 10) Shyly, she asks, "I mean, Daddy, is there?"
- 逻辑非查询，使用 ~ 运算符得到不匹配查询条件的所有行：
 Executing Query for: ~(Alice)
 ~(Alice) occurs 9 times
 (line 2) Her Daddy says when the wind blows
 (line 3) through her hair, it looks almost alive,
 (line 4) like a fiery bird in flight.
 ...
- 逻辑或查询，使用 | 运算符返回匹配两个条件中任意一个的行：
 Executing Query for: (hair | Alice)
 (hair | Alice) occurs 2 times
 (line 1) Alice Emma has long flowing red hair.
 (line 3) through her hair, it looks almost alive,
- 逻辑与查询，使用 & 运算符返回匹配全部两个条件的行：
 Executing query for: (hair & Alice)
 (hair & Alice) occurs 1 time
 (line 1) Alice Emma has long flowing red hair.

此外，我们还希望能够混合使用这些运算符，比如：

```
fiery & bird | wind
```

在类似这样的例子中，我们将使用 C++ 通用的优先级规则（参见 4.1.2 节，第 121 页）对复杂表达式求值。因此，这条查询语句所得行应该是如下二者之一：在该行中或者 fiery 和 bird 同时出现，或者出现了 wind：

```
Executing Query for: ((fiery & bird) | wind)
((fiery & bird) | wind) occurs 3 times
(line 2) Her Daddy says when the wind blows
(line 4) like a fiery bird in flight.
(line 5) A beautiful fiery bird, he tells her,
```

在输出内容中首先是那条查询语句，我们使用圆括号来表示查询被解释和执行的次序。与之前实现的版本一样，接下来系统将按照查询结果中行号的升序显示结果并且每一行只显示一次。

15.9.1 面向对象的解决方案

我们可能会认为使用 12.3.2 节（第 432 页）的 TextQuery 类来表示单词查询，然后

从该类中派生出其他查询是一种可行的方案。

然而，这样的设计实际上存在缺陷。为了理解其中的原因，我们不妨考虑逻辑非查询。单词查询查找一个指定的单词，为了让逻辑非查询按照单词查询的方式执行，我们将不得不定义逻辑非查询所要查找的单词。但是在一般情况下，我们无法得到这样的单词。相反，一个逻辑非查询中含有一个结果值需要取反的查询语句（单词查询或任何其他查询）；类似的，一个逻辑与查询和一个逻辑或查询各包含两个结果值需要合并的查询语句。

由上述观察结果可知，我们应该将几种不同的查询建模成相互独立的类，这些类共享一个公共基类：

```
WordQuery      // Daddy
NotQuery       // ~Alice
OrQuery        // hair | Alice
AndQuery       // hair & Alice
```

这些类将只包含两个操作：

- eval, 接受一个 TextQuery 对象并返回一个 QueryResult, eval 函数使用给定的 TextQuery 对象查找与之匹配的行。
- rep, 返回基础查询的 string 表示形式, eval 函数使用 rep 创建一个表示匹配结果的 QueryResult, 输出运算符使用 rep 打印查询表达式。

637

关键概念：继承与组合

继承体系的设计本身是一个非常复杂的问题，已经超出了本书的范围。然而，有一条设计准则非常重要也非常基础，每个程序员都应该熟悉它。

当我们令一个类公有地继承另一个类时，派生类应当反映与基类的“是一种 (Is A)”关系。在设计良好的类体系当中，公有派生类的对象应该可以用在任何需要基类对象的地方。

类型之间的另一种常见关系是“有一个 (Has A)”关系，具有这种关系的类暗含成员的意思。

在我们的书店示例中，基类表示的是按规定价格销售的书籍的报价。Bulk_quote “是一种”报价结果，只不过它使用的价格策略不同。我们的书店类都“有一个”价格成员和 ISBN 成员。

抽象基类

如我们所知，在这四种查询之间并不存在彼此的继承关系，从概念上来说它们互为兄弟。因为所有这些类都共享同一个接口，所以我们需要定义一个抽象基类（参见 15.4 节，第 541 页）来表示该接口。我们将所需的抽象基类命名为 Query_base，以此来表示它的角色是整个查询继承体系的根节点。

我们的 Query_base 类将把 eval 和 rep 定义成纯虚函数（参见 15.4 节，第 541 页），其他代表某种特定查询类型的类必须覆盖这两个函数。我们将从 Query_base 直接派生出 WordQuery 和 NotQuery。AndQuery 和 OrQuery 都具有系统中其他类所不具备的一个特殊属性：它们各自包含两个运算对象。为了对这种属性建模，我们定义另外一个名为 BinaryQuery 的抽象基类，该抽象基类用于表示含有两个运算对象的查询。AndQuery 和 OrQuery 继承自 BinaryQuery，而 BinaryQuery 继承自 Query_base。由这些分

析我们将得到如图 15.2 所示的类设计结果:

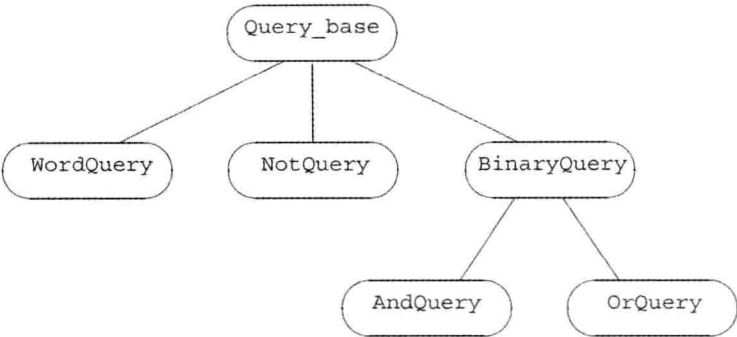


图 15.2: Query_base 继承体系

将层次关系隐藏于接口类中

我们的程序将致力于计算查询结果,而非仅仅构建查询的体系。为了使程序能正常运行,我们必须首先创建查询命令,最简单的办法是编写 C++表达式。例如,可以编写下面的代码来生成之前描述的复合查询:

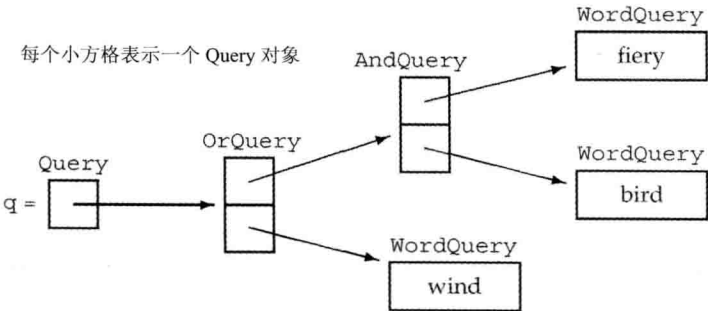
```
Query q = Query("fiery") & Query("bird") | Query("wind");
```

如上所述,其隐含的意思是用户层代码将不会直接使用这些继承的类;相反,我们将定义一个名为 Query 的接口类,由它负责隐藏整个继承体系。Query 类将保存一个 Query_base 指针,该指针绑定到 Query_base 的派生类对象上。Query 类与 Query_base 类提供的操作是相同的:eval 用于求查询的结果,rep 用于生成查询的 string 版本,同时 Query 也会定义一个重载的输出运算符用于显示查询。

◀ 638

用户将通过 Query 对象的操作间接地创建并处理 Query_base 对象。我们定义 Query 对象的三个重载运算符以及一个接受 string 参数的 Query 构造函数,这些函数动态分配一个新的 Query_base 派生类的对象:

- &运算符生成一个绑定到新的 AndQuery 对象上的 Query 对象;
- |运算符生成一个绑定到新的 OrQuery 对象上的 Query 对象;
- ~运算符生成一个绑定到新的 NotQuery 对象上的 Query 对象;
- 接受 string 参数的 Query 构造函数生成一个新的 WordQuery 对象。



由表达式创建的对象

```
Query q = Query("fiery") & Query("bird") | Query("wind");
```

图 15.3: 使用 Query 表达式创建的对象

理解这些类的工作机理

在这个应用程序中，很大一部分工作是构建代表用户查询的对象，对于读者来说认识到这一点非常重要。例如，像上面这样的表达式将生成如图 15.3 所示的一系列相关对象的集合。

一旦对象树构建完成后，对某一条查询语句的求值（或生成表示形式的）过程基本上就转换为沿着箭头方向依次对每个对象求值（或显示）的过程（由编译器为我们组织管理）。
639 例如，如果我们对 q（即树的根节点）调用 eval 函数，则该调用语句将令 q 所指的 OrQuery 对象 eval 它自己。对该 OrQuery 求值实际上是对它的两个运算对象执行 eval 操作：一个运算对象是 AndQuery，另一个是查找单词 wind 的 WordQuery。接下来，对 AndQuery 求值转化为对它的两个 WordQuery 求值，分别生成单词 fiery 和 bird 的查询结果。

对于面向对象编程的新手来说，要想理解一个程序，最困难的部分往往是理解程序的设计思路。一旦你掌握了程序的设计思路，接下来的实现也就水到渠成了。为了帮助读者理解程序设计的过程，我们在表 15.1 中整理了之前那个例子用到的类，并对其进行了简要的描述。

640

表 15.1：概述：Query 程序设计

Query 程序接口类和操作	
TextQuery	该类读入给定的文件并构建一个查找图。这个类包含一个 query 操作，它接受一个 string 实参，返回一个 QueryResult 对象；该 QueryResult 对象表示 string 出现的行（12.3.2 节，第 432 页）
QueryResult	该类保存一个 query 操作的结果（12.3.2 节，第 433 页）
Query	是一个接口类，指向 Query_base 派生类的对象
Query q(s)	将 Query 对象 q 绑定到一个存放着 strings 的新 WordQuery 对象上
q1 & q2	返回一个 Query 对象，该 Query 绑定到一个存放 q1 和 q2 的新 AndQuery 对象上
q1 q2	返回一个 Query 对象，该 Query 绑定到一个存放 q1 和 q2 的新 OrQuery 对象上
~q	返回一个 Query 对象，该 Query 绑定到一个存放 q 的新 NotQuery 对象上
Query 程序实现类	
Query_base	查询类的抽象基类
WordQuery	Query_base 的派生类，用于查找一个给定的单词
NotQuery	Query_base 的派生类，查询结果是 Query 运算对象没有出现的行的集合
BinaryQuery	Query_base 派生出来的另一个抽象基类，表示有两个运算对象的查询
OrQuery	BinaryQuery 的派生类，返回它的两个运算对象分别出现的行的并集
AndQuery	BinaryQuery 的派生类，返回它的两个运算对象分别出现的行的交集

15.9.1 节练习

练习 15.31：已知 s1、s2、s3 和 s4 都是 string，判断下面的表达式分别创建了什么样的对象：

(a) Query(s1) | Query(s2) & ~ Query(s3);

(b) Query(s1) | (Query(s2) & ~ Query(s3));

(c) (Query(s1) & (Query(s2))) | (Query(s3) & Query(s4));

15.9.2 Query_base 类和 Query 类

下面我们开始程序的实现过程，首先定义 Query_base 类：

```
// 这是一个抽象基类，具体的查询类型从中派生，所有成员都是 private 的
class Query_base {
    friend class Query;
protected:
    using line_no = TextQuery::line_no; // 用于 eval 函数
    virtual ~Query_base() = default;
private:
    // eval 返回与当前 Query 匹配的 QueryResult
    virtual QueryResult eval(const TextQuery&) const = 0;
    // rep 是表示查询的一个 string
    virtual std::string rep() const = 0;
};
```

eval 和 rep 都是纯虚函数，因此 Query_base 是一个抽象基类（参见 15.4 节，第 541 页）。因为我们不希望用户或者派生类直接使用 Query_base，所以它没有 public 成员。所有对 Query_base 的使用都需要通过 Query 对象，因为 Query 需要调用 Query_base 的虚函数，所以我们将 Query 声明成 Query_base 的友元。

受保护的成员 line_no 将在 eval 函数内部使用。类似的，析构函数也是受保护的，因为它将（隐式地）在派生类析构函数中使用。

Query 类

Query 类对外提供接口，同时隐藏了 Query_base 的继承体系。每个 Query 对象都含有一个指向 Query_base 对象的 shared_ptr。因为 Query 是 Query_base 的唯一接口，所以 Query 必须定义自己的 eval 和 rep 版本。

接受一个 string 参数的 Query 构造函数将创建一个新的 WordQuery 对象，然后将它的 shared_ptr 成员绑定到这个新创建的对象上。&、| 和 ~ 运算符分别创建 AndQuery、OrQuery 和 NotQuery 对象，这些运算符将返回一个绑定到新创建的对象上的 Query 对象。为了支持这些运算符，Query 还需要另外一个构造函数，它接受指向 Query_base 的 shared_ptr 并且存储给定的指针。我们将这个构造函数声明为私有的，原因是我们不希望一般的用户代码能随便定义 Query_base 对象。因为这个构造函数是私有的，所以我们需要将三个运算符声明为友元。

在形成了上述设计思路后，Query 类本身就比较简单了：

```
// 这是一个管理 Query_base 继承体系的接口类
class Query {
    // 这些运算符需要访问接受 shared_ptr 的构造函数，而该函数是私有的
    friend Query operator~(const Query &);
    friend Query operator|(const Query&, const Query&);
    friend Query operator&(const Query&, const Query&);
public:
    Query(const std::string&); // 构建一个新的 WordQuery
    // 接口函数：调用对应的 Query_base 操作
    QueryResult eval(const TextQuery &t) const
    { return q->eval(t); }
    std::string rep() const { return q->rep(); }
```



```
private:
    Query(std::shared_ptr<Query_base> query): q(query) { }
    std::shared_ptr<Query_base> q;
};
```

我们首先将创建 Query 对象的运算符声明为友元，之所以这么做是因为这些运算符需要访问那个私有构造函数。

在 Query 的公有接口部分，我们声明了接受 string 的构造函数，不过没有对其进行定义。因为这个构造函数将要创建一个 WordQuery 对象，所以我们应该首先定义 WordQuery 类，随后才能定义接受 string 的 Query 构造函数。

另外两个公有成员是 Query_base 的接口。其中，Query 操作使用它的 Query_base 指针来调用各自的 Query_base 虚函数。实际调用哪个函数版本将由 q 所指的对象类型决定，并且直到运行时才能最终确定下来。



Query 的输出运算符

输出运算符可以很好地解释我们的整个查询系统是如何工作的：

```
std::ostream &
operator<<(std::ostream &os, const Query &query)
{
    // Query::rep 通过它的 Query_base 指针对 rep() 进行了虚调用
    return os << query.rep();
}
```

当我们打印一个 Query 时，输出运算符调用 Query 类的公有 rep 成员。运算符函数通过指针成员虚调用当前 Query 所指对象的 rep 成员。也就是说，当我们编写如下代码时：

```
Query andq = Query(sought1) & Query(sought2);
cout << andq << endl;
```

输出运算符将调用 andq 的 Query::rep，而 Query::rep 通过它的 Query_base 指针虚调用 Query_base 版本的 rep 函数。因为 andq 指向的是一个 AndQuery 对象，所以本次的函数调用将运行 AndQuery::rep。

15.9.2 节练习

练习 15.32: 当一个 Query 类型的对象被拷贝、移动、赋值或销毁时，将分别发生什么？

练习 15.33: 当一个 Query_base 类型的对象被拷贝、移动、赋值或销毁时，将分别发生什么？

642 15.9.3 派生类

对于 Query_base 的派生类来说，最有趣的部分是这些派生类如何表示一个真实的查询。其中 WordQuery 类最直接，它的任务就是保存要查找的单词。

其他类分别操作一个或两个运算对象。NotQuery 有一个运算对象，AndQuery 和 OrQuery 有两个。在这些类当中，运算对象可以是 Query_base 的任意一个派生类的对象：一个 NotQuery 对象可以被用在 WordQuery、AndQuery、OrQuery 或另一个 NotQuery 中。为了支持这种灵活性，运算对象必须以 Query_base 指针的形式存储，

这样我们就能把该指针绑定到任何我们需要的具体类上。

然而，实际上我们的类并不存储 `Query_base` 指针，而是直接使用一个 `Query` 对象。就像用户代码可以通过接口类得到简化一样，我们也可以使用接口类来简化我们自己的类。

至此我们已经清楚了所有类的设计思路，接下来依次实现它们。

WordQuery 类

一个 `WordQuery` 查找一个给定的 `string`，它是在给定的 `TextQuery` 对象上实际执行查询的唯一一个操作：

```
class WordQuery: public Query_base {
    friend class Query;                // Query 使用 WordQuery 构造函数
    WordQuery(const std::string &s): query_word(s) { }
    // 具体的类: WordQuery 将定义所有继承而来的纯虚函数
    QueryResult eval(const TextQuery &t) const
    { return t.query(query_word); }
    std::string rep() const { return query_word; }
    std::string query_word;           // 要查找的单词
};
```

和 `Query_base` 一样，`WordQuery` 没有公有成员。同时，`Query` 必须作为 `WordQuery` 的友元，这样 `Query` 才能访问 `WordQuery` 的构造函数。

每个表示具体查询的类都必须定义继承而来的纯虚函数 `eval` 和 `rep`。我们在 `WordQuery` 类的内部定义这两个操作：`eval` 调用其 `TextQuery` 参数的 `query` 成员，由 `query` 成员在文件中实际进行查找；`rep` 返回这个 `WordQuery` 表示的 `string`（即 `query_word`）。

定义了 `WordQuery` 类之后，我们就能定义接受 `string` 的 `Query` 构造函数了：

```
inline
Query::Query(const std::string &s): q(new WordQuery(s)) { }
```

这个构造函数分配一个 `WordQuery`，然后令其指针成员指向新分配的对象。

NotQuery 类及 ~ 运算符

~ 运算符生成一个 `NotQuery`，其中保存着一个需要对其取反的 `Query`：

```
class NotQuery: public Query_base {
    friend Query operator~(const Query &);
    NotQuery(const Query &q): query(q) { }
    // 具体的类: NotQuery 将定义所有继承而来的纯虚函数
    std::string rep() const {return "~(" + query.rep() + ")";}
    QueryResult eval(const TextQuery&) const;
    Query query;
};

inline Query operator~(const Query &operand)
{
    return std::shared_ptr<Query_base>(new NotQuery(operand));
}
```

643

因为 `NotQuery` 的所有成员都是私有的，所以我们一开始就要把 ~ 运算符设定为友元。为

了 rep 一个 NotQuery，我们需要将~符号与基础的 Query 连接在一起。我们在输出的结果中加上适当的括号，这样读者就可以清楚地知道查询的优先级了。

值得注意的是，在 NotQuery 自己的 rep 成员中对 rep 的调用最终执行的是一个虚调用：query.rep() 是对 Query 类 rep 成员的非虚调用，接着 Query::rep 将调用 q->rep()，这是一个通过 Query_base 指针进行的虚调用。

~运算符动态分配一个新的 NotQuery 对象，其 return 语句隐式地使用接受一个 shared_ptr<Query_base> 的 Query 构造函数。也就是说，return 语句等价于：

```
// 分配一个新的 NotQuery 对象
// 将所得的 NotQuery 指针绑定到一个 shared_ptr<Query_base>
shared_ptr<Query_base> tmp(new NotQuery(expr));
return Query(tmp);           // 使用接受一个 shared_ptr 的 Query 构造函数
```

eval 成员比较复杂，因此我们将在类的外部实现它，15.9.4 节（第 573 页）将专门介绍如何定义 eval 函数。

BinaryQuery 类

BinaryQuery 类也是一个抽象基类，它保存操作两个运算对象的查询类型所需的数据：

```
class BinaryQuery: public Query_base {
protected:
    BinaryQuery(const Query &l, const Query &r, std::string s):
        lhs(l), rhs(r), opSym(s) { }
    // 抽象类: BinaryQuery 不定义 eval
    std::string rep() const { return "(" + lhs.rep() + " "
                                   + opSym + " "
                                   + rhs.rep() + ")"; }
    Query lhs, rhs;           // 左侧和右侧运算对象
    std::string opSym;         // 运算符的名字
};
```

644 BinaryQuery 中的数据是两个运算对象及相应的运算符符号，构造函数负责接受两个运算对象和一个运算符符号，然后将它们存储在对应的数据成员中。

要想 rep 一个 BinaryQuery，我们需要生成一个带括号的表达式。表达式的内容依次包括左侧运算对象、运算符以及右侧运算对象。就像我们显示 NotQuery 的方法一样，对 rep 的调用最终是对 lhs 和 rhs 所指 Query_base 对象的 rep 函数进行虚调用。



BinaryQuery 不定义 eval，而是继承了该纯虚函数。因此，BinaryQuery 也是一个抽象基类，我们不能创建 BinaryQuery 类型的对象。

AndQuery 类、OrQuery 类及相应的运算符

AndQuery 类和 OrQuery 类以及它们的运算符都非常相似：

```
class AndQuery: public BinaryQuery {
    friend Query operator&(const Query&, const Query&);
    AndQuery(const Query &left, const Query &right):
        BinaryQuery(left, right, "&") { }
    // 具体的类: AndQuery 继承了 rep 并且定义了其他纯虚函数
    QueryResult eval(const TextQuery&) const;
```

```

};
inline Query operator&(const Query &lhs, const Query &rhs)
{
    return std::shared_ptr<Query_base>(new AndQuery(lhs, rhs));
}

class OrQuery: public BinaryQuery {
    friend Query operator|(const Query&, const Query&);
    OrQuery(const Query &left, const Query &right):
        BinaryQuery(left, right, "|") { }
    QueryResult eval(const TextQuery&) const;
};
inline Query operator|(const Query &lhs, const Query &rhs)
{
    return std::shared_ptr<Query_base>(new OrQuery(lhs, rhs));
}

```

这两个类将各自的运算符定义成友元，并且各自定义了一个构造函数通过运算符创建 BinaryQuery 基类部分。它们继承 BinaryQuery 的 rep 函数，但是覆盖了 eval 函数。

和 ~ 运算符一样，& 和 | 运算符也返回一个绑定到新分配对象上的 shared_ptr。在这些运算符中，return 语句负责将 shared_ptr 转换成 Query。

15.9.3 节练习

645

练习 15.34: 针对图 15.3（第 565 页）构建的表达式：

- (a) 列举出在处理表达式的过程中执行的所有构造函数。
- (b) 列举出 cout<<q 所调用的 rep。
- (c) 列举出 q.eval() 所调用的 eval。

练习 15.35: 实现 Query 类和 Query_base 类，其中需要定义 rep 而无须定义 eval。

练习 15.36: 在构造函数和 rep 成员中添加打印语句，运行你的代码以检验你对本节第一个练习中 (a)、(b) 两小题的回答是否正确。

练习 15.37: 如果在派生类中含有 shared_ptr<Query_base> 类型的成员而非 Query 类型的成员，则你的类需要做出怎样的改变？

练习 15.38: 下面的声明合法吗？如果不合法，请解释原因；如果合法，请指出该声明的含义。

```

BinaryQuery a = Query("fiery") & Query("bird");
AndQuery b = Query("fiery") & Query("bird");
OrQuery c = Query("fiery") & Query("bird");

```

15.9.4 eval 函数

eval 函数是我们这个查询系统的核心。每个 eval 函数作用于各自的运算对象，同时遵循的内在逻辑也有所区别：OrQuery 的 eval 操作返回两个运算对象查询结果的并集，而 AndQuery 返回交集。与它们相比，NotQuery 的 eval 函数更加复杂一些：它需要返回运算对象没有出现的文本行。

为了支持上述 `eval` 函数的处理, 我们需要使用 `QueryResult`, 在它当中定义了 12.3.2 节练习 (第 435 页) 添加的成员。假设 `QueryResult` 包含 `begin` 和 `end` 成员, 它们允许我们在 `QueryResult` 保存的行号 `set` 中进行迭代; 另外假设 `QueryResult` 还包含一个名为 `get_file` 的成员, 它返回一个指向待查询文件的 `shared_ptr`。



我们的 `Query` 类使用了 12.3.2 节练习 (第 435 页) 为 `QueryResult` 定义的成员。

OrQuery::eval

一个 `OrQuery` 表示的是它的两个运算对象结果的并集, 对于每个运算对象来说, 我们通过调用 `eval` 得到它的查询结果。因为这些运算对象的类型是 `Query`, 所以调用 `eval` 也就是调用 `Query::eval`, 而后者实际上是对潜在的 `Query_base` 对象的 `eval` 进行虚调用。每次调用完成后, 得到的结果是一个 `QueryResult`, 它表示运算对象出现的行号。我们把这些行号组织在一个新 `set` 中:

646

```
// 返回运算对象查询结果 set 的并集
QueryResult
OrQuery::eval(const TextQuery& text) const
{
    // 通过 Query 成员 lhs 和 rhs 进行的虚调用
    // 调用 eval 返回每个运算对象的 QueryResult
    auto right = rhs.eval(text), left = lhs.eval(text);
    // 将左侧运算对象的行号拷贝到结果 set 中
    auto ret_lines =
        make_shared<set<line_no>>(left.begin(), left.end());
    // 插入右侧运算对象所得的行号
    ret_lines->insert(right.begin(), right.end());
    // 返回一个新的 QueryResult, 它表示 lhs 和 rhs 的并集
    return QueryResult(rep(), ret_lines, left.get_file());
}
```

我们使用接受一对迭代器的 `set` 构造函数初始化 `ret_lines`。一个 `QueryResult` 的 `begin` 和 `end` 成员返回行号 `set` 的迭代器, 因此, 创建 `ret_lines` 的过程实际上是拷贝了 `left` 集合的元素。接下来对 `ret_lines` 调用 `insert`, 并将 `right` 的元素插入进来。调用结束后, `ret_lines` 将包含在 `left` 或 `right` 中出现过的所有行号。

`eval` 函数在最后构建并返回一个表示混合查询匹配的 `QueryResult`。`QueryResult` 的构造函数 (参见 12.3.2 节, 第 434 页) 接受三个实参: 一个表示查询的 `string`、一个指向匹配行号 `set` 的 `shared_ptr` 和一个指向输入文件 `vector` 的 `shared_ptr`。我们调用 `rep` 生成所需的 `string`, 调用 `get_file` 获取指向文件的 `shared_ptr`。因为 `left` 和 `right` 指向的是同一个文件, 所以使用哪个执行 `get_file` 函数并不重要。

AndQuery::eval

`AndQuery` 的 `eval` 和 `OrQuery` 很类似, 唯一的区别是它调用了一个标准库算法来求得两个查询结果中共有的行:

```
// 返回运算对象查询结果 set 的交集
QueryResult
AndQuery::eval(const TextQuery& text) const
{

```

```

// 通过 Query 运算对象进行的虚调用, 以获得运算对象的查询结果 set
auto left = lhs.eval(text), right = rhs.eval(text);
// 保存 left 和 right 交集的 set
auto ret_lines = make_shared<set<line_no>>();
// 将两个范围的交集写入一个目的迭代器中
// 本次调用的目的迭代器向 ret 添加元素
set_intersection(left.begin(), left.end(),
                 right.begin(), right.end(),
                 inserter(*ret_lines, ret_lines->begin()));
return QueryResult(rep(), ret_lines, left.get_file());
}

```

其中我们使用标准库算法 `set_intersection` 来合并两个 `set`, 关于 `set_intersection` 在附录 A.2.8 (第 779 页) 中有详细的描述。 647

`set_intersection` 算法接受五个迭代器。它使用前四个迭代器表示两个输入序列 (参见 10.5.2 节, 第 368 页), 最后一个实参表示目的位置。该算法将两个输入序列中共同出现的元素写入到目的位置中。

在上述调用中我们传入一个插入迭代器 (参见 10.4.1 节, 第 357 页) 作为目的位置。当 `set_intersection` 向这个迭代器写入内容时, 实际上是向 `ret_lines` 插入一个新元素。

和 `OrQuery` 的 `eval` 函数一样, `AndQuery` 的 `eval` 函数也在最后构建并返回一个表示混合查询匹配的 `QueryResult`。

NotQuery::eval

`NotQuery` 查找运算对象没有出现的文本行:

```

// 返回运算对象的结果 set 中不存在的行
QueryResult
NotQuery::eval(const TextQuery& text) const
{
    // 通过 Query 运算对象对 eval 进行虚调用
    auto result = query.eval(text);
    // 开始时结果 set 为空
    auto ret_lines = make_shared<set<line_no>>();
    // 我们必须在运算对象出现的所有行中进行迭代
    auto beg = result.begin(), end = result.end();
    // 对于输入文件的每一行, 如果该行不在 result 当中, 则将其添加到 ret_lines
    auto sz = result.get_file()->size();
    for (size_t n = 0; n != sz; ++n) {
        // 如果我们还没有处理完 result 的所有行
        // 检查当前行是否存在
        if (beg == end || *beg != n)
            ret_lines->insert(n);    // 如果不在 result 当中, 添加这一行
        else if (beg != end)
            ++beg;                  // 否则继续获取 result 的下一行 (如果有的话)
    }
    return QueryResult(rep(), ret_lines, result.get_file());
}

```

和其他 `eval` 函数一样, 我们首先对当前的运算对象调用 `eval`, 所得的结果

QueryResult 中包含的是运算对象出现的行号,但我们想要的是运算对象未出现的行号。也就是说,我们需要的是存在于文件中,但是不在 result 中的行。

要想得到最终的结果,我们需要遍历不超过输出文件大小的所有整数,并将所有不在 result 中的行号放入到 ret_lines 中。我们使用 beg 和 end 分别表示 result 的第一个元素和最后一个元素的下一位置。因为遍历的对象是一个 set,所以当遍历结束后获得的行号将按照升序排列。

648

循环体负责检查当前的编号是否在 result 当中。如果不在,将这个数字添加到 ret_lines 中;如果该数字属于 result,则我们递增 result 的迭代器 beg。

一旦处理完所有行号,就返回包含 ret_lines 的一个 QueryResult 对象;和之前版本的 eval 类似,该 QueryResult 对象还包含 rep 和 get_file 的运行结果。

15.9.4 节练习

练习 15.39: 实现 Query 类和 Query_base 类,求图 15.3 (第 565 页) 中表达式的值并打印相关信息,验证你的程序是否正确。

练习 15.40: 在 OrQuery 的 eval 函数中,如果 rhs 成员返回的是空集将发生什么?如果 lhs 是空集呢?如果 lhs 和 rhs 都是空集又将发生什么?

练习 15.41: 重新实现你的类,这次使用指向 Query_base 的内置指针而非 shared_ptr。请注意,做出上述改动后你的类将不能再使用合成的拷贝控制成员。

练习 15.42: 从下面的几种改进中选择一种,设计并实现它:

- (a) 按句子查询并打印单词,而不再是按行打印。
- (b) 引入一个历史系统,用户可以按编号查阅之前的某个查询,并可以在其中增加内容或者将其与其他查询组合。
- (c) 允许用户对结果做出限制,比如从给定范围的行中挑出匹配的进行显示。

小结

649

继承使得我们可以编写一些新的类，这些新类既能共享其基类的行为，又能根据需要覆盖或添加行为。动态绑定使得我们可以忽略类型之间的差异，其机理是在运行时根据对象的动态类型来选择运行函数的哪个版本。继承和动态绑定的结合使得我们能够编写具有特定类型行为但又独立于类型的程序。

在 C++ 语言中，动态绑定只作用于虚函数，并且需要通过指针或引用调用。

在派生类对象中包含有与它的每个基类对应的子对象。因为所有派生类对象都含有基类部分，所以我们将派生类的引用或指针转换为一个可访问的基类引用或指针。

当执行派生类的构造、拷贝、移动和赋值操作时，首先构造、拷贝、移动和赋值其中的基类部分，然后才轮到派生类部分。析构函数的执行顺序则正好相反，首先销毁派生类，接下来执行基类子对象的析构函数。基类通常都应该定义一个虚析构函数，即使基类根本不需要析构函数也最好这么做。将基类的析构函数定义成虚函数的原因是为了确保当我们删除一个基类指针，而该指针实际指向一个派生类对象时，程序也能正确运行。

派生类为它的每个基类提供一个保护级别。public 基类的成员也是派生类接口的一部分；private 基类的成员是不可访问的；protected 基类的成员对于派生类的派生类是可访问的，但是对于派生类的用户不可访问。

术语表

抽象基类 (abstract base class) 含有一个或多个纯虚函数的类，我们无法创建抽象基类的对象。

可访问的 (accessible) 能被派生类对象访问的基类成员。可访问性由派生类的派生列表中所用的访问说明符和基类中成员的访问级别共同决定。例如，通过公有继承而来的一个公有成员对于派生类的用户来说是可访问的；而私有继承而来的公有成员是不可访问的。

基类 (base class) 可供其他类继承的类。基类的成员也将成为派生类的成员。

类派生列表 (class derivation list) 罗列了所有基类，每个基类包含一个可选的访问级别，它定义了派生类继承该基类的方式。如果没有提供访问说明符，则当派生类通过关键字 struct 定义时继承是公有的；而当派生类通过关键字 class 定义时继承是私有的。

派生类 (derived class) 从其他类派生而

来的类。派生类可以覆盖其基类的虚函数，也可以定义自己的新成员。派生类的作用域嵌套在基类作用域当中；派生类的成员能直接访问基类的成员。

派生类向基类的类型转换 (derived-to-base conversion) 派生类对象向基类引用或者派生类指针向基类指针的隐式类型转换。

直接基类 (direct base class) 派生类直接继承的基类，直接基类在派生类的派生列表中说明。直接基类本身也可以是一个派生类。

动态绑定 (dynamic binding) 直到运行时才确定到底执行函数的哪个版本。在 C++ 语言中，动态绑定的意思是在运行时根据引用或指针所绑定对象的实际类型来选择执行虚函数的某一个版本。

动态类型 (dynamic type) 对象在运行时的类型。引用所引对象或者指针所指对象的动态类型可能与该引用或指针的静态类型不同。基类的指针或引用可以指向一个

650